Closebook — Product Requirements (TestSprite copy)

Closebook closes the books in real time for a cross-border SME. Every time money lands it matches the inflow to open invoices, then makes the money decision that match unlocks: release or hold supplier payments, convert only the FX it needs, chase shortfalls, and escalate anything unclear to a person.

One principle governs everything below: the model proposes, deterministic code enforces. Amounts, limits, reserve floors, approvals, idempotency and duplicate protection live in code, never in prompts. A model may read a messy reference and suggest which invoice it belongs to. It cannot move money, and no number it produces is used without being re-derived and re-checked in code.

Runs against the Airwallex sandbox. No real money moves.

---

1. What it does

- **Money in.** A deposit arrives with whatever the payer typed — a name, a fragment of an invoice number, a typo. Closebook reads it through five tiers in order: an exact reference match, a rule (known payer plus a plausible amount), a subset-sum across several invoices, a best-fit part-payment, and finally a model reading. Every model proposal is re-checked arithmetically.
- **The decision.** Matching an invoice is not permission to spend. The planner answers what the money may do against a committed-cash position, a reserve floor and an automatic payment limit.
- **Money out.** Payments go out through one module that persists a request id before anything leaves, never re-sends a step that already has a bank reference, and re-verifies every approval at execution time.

---

2. Confidence and the auto band

Matching runs on integer minor units and produces a confidence between 0 and 1.

- **At or above 0.90** the reading may be applied automatically.
- **Below 0.90 it is a PROPOSAL, not an application.** The money stays in

suspense, the invoice is untouched, and nothing is posted against it until a

person approves.

- **A partial payment below 0.90 is HELD FOR APPROVAL.** It is not applied on

the strength of an amount that merely looks close, and it is not dropped: it

appears in **Needs your OK** with the invoice it claims and its confidence.

- **Overpayments and part-payments are deliberately capped below 0.90** so they

can never auto-apply. An overpayment creates a refund obligation and a

part-payment needs a human to decide whether to chase.

- **EXACT means a clean reference match only.** A folded `0/0` is a RULE at a

lower confidence, not an exact match.

- A model's own claim is capped below the auto band before it can matter. If a

model claims 0.90 the code pins it to 0.89 and holds it for a person.

Suspense can leave three ways, each a deliberate decision with a PIN: match it to

an invoice, record a refund owed (the cash stays put and stops being spendable),

or release it as income with a note.

---

3. Holds, and how they differ

Every payable that is not being paid carries **all** the reasons it is not, not

just the first, because two holds need two different actions.

Hold	What it means	What fixes it
Dependency	**A bill waits on an invoice that has not fully landed**	**The customer pays**
Reserve floor	Paying would take committed cash below the floor	More cash, or an approval
Auto-pay limit	The amount exceeds the limit a human must agree to	An approval

Capacity	The currency is not held and cannot be bought	Funds, or a different rail

A dependency hold is not a reserve hold. They read similarly on a screen and need opposite responses: a dependency hold clears itself when the customer pays and no approval can release it, while a reserve hold clears the moment a person approves it or cash arrives. A part-payment does **not** release a dependent payable — the dependency is met only by a fully paid receivable.

An over-limit bill that is otherwise payable **must produce an approval card in Needs your OK**. A bill blocked for another reason as well shows** both** reasons and stays in **Held — and not waiting on you**, because a hold that no person can lift does not belong in a queue of things for a person to do.

---

4. Payments and their states

```
Planned → Checked → Approved → Sent → Confirmed
```

A payment that did not complete shows a **red X at the step where it stopped**.

- **Only FAILED, RETURNED and CANCELLED get the X.** A healthy in-flight transfer never does, however many earlier attempts on that bill failed.
- **A returned transfer ends CANCELLED, with its reason shown next to it.** The bank reports a return as CANCELLED, which on its own reads as "someone stopped this". The reason the bank gave is the point of the row.
- **Between approval and a bank reference the card reads "Sending to the bank…"**, not APPROVED with an empty reference line.
- **A replacement is a new operation**, not a retry: a new request id and a fresh approval, because the original terms were approved for a payment that did not happen.

---

5. Sandbox controls

The deposit simulator and the **Settle / Fail / Return** buttons live in one **collapsed, dashed-border panel labelled "sandbox only"**. Fail and Return ask for confirmation. Each row names its payment ("PO-7001 · Atlanta Packaging · USD 1,800") and carries a per-transfer test id, and a payment card that has a control links to its row in the panel.

The normal screens show none of it. Home, AI activity and Setup contain no sandbox control at all.

Once a transfer is terminal there is nothing to simulate, and the controls are not offered.

---

6. Model said / Code decided

Every model call and every money-in payment carries a two-column receipt:

- **Model said** — the invoice, the amount, the confidence it claimed, one line of reasoning. Where the code overrode it, the claimed value is struck through.
- **Code decided** — the verdict, the rule that fired, and the final confidence.

The final confidence comes from the **stored decision**, never from the model's claim. When the code keeps the deterministic reading over the model, the final confidence is the deterministic one (for example 0.85) and the rule that fired is RULE.

When the model was unavailable the left column says so in plain words ("Model unavailable, so this went to a person instead of guessing.") and the right column shows the escalation.

---

7. What is never shown

- **No raw vendor errors.** A refused sandbox call reads "Airwallex's sandbox

refused this step. Nothing changed in your books. Try again in a minute." The machine's own words are kept behind a **Detail** toggle, including for failures recorded days earlier.

- **No invented evidence.** Every figure on a screen is counted from the book.
- **No unbounded waits.** Every action resolves to a success or a plain error with a timeout; a button never stays on "Working…".

---

8. Testing in a shared workspace

Several test runs share one sandbox account and one Airwallex key. Two instances of the app can see each other's deposits. Follow these rules so one run does not corrupt another's evidence.

Every record gets a unique suffix. Names, invoice numbers, payable numbers and references must be unique per run — for example `QA-OVERLIMIT-7f31`, not `QA-OVERLIMIT-001`. Two runs using the same bill number will match each other's payments and neither result means anything.

Never assume another test's data or settings. Do not assume a customer, supplier, invoice or payable exists because a previous run created one, and do not assume the reserve floor, auto-pay limit or committed cash is what you set unless this run set it. Read the settings you depend on and assert on what you read.

Read balances before and after. Take the cash position, suspense, inflow counts and incident counts before the action, then again after, and assert on the **difference**. An absolute figure is a claim about the whole workspace, including everyone else's activity; a delta is a claim about your action.

Count what you found before asserting on it. A parser or a page scan that matches zero rows and then asserts nothing about them passes over a broken parser.

Clean up by identity, not by wholesale reset. Delete the records this run

created, identified by their unique suffix. A reset that clears the workspace

destroys another run's evidence.

---

9. Out of scope

- Real money. This is the sandbox.
- Card issuing: the sandbox rejects issuance for this account, so the card beat

is a policy verdict rather than a live card.

- Multi-user sign-in. A single operator PIN gates every write.